

AIQT Milestone 15 Build Specification v0.2

M15 - Source Control, GitHub Flow, and Work Unit Release Discipline

Status: Ready for implementation review after Claude validation. This v0.2 resolves the v0.1 review findings around risk score alignment, self-attested reporting, packet section ordering, source alignment, and schema-file path flexibility.

1. Source Alignment

This milestone extends the AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3. It preserves the governing pipeline and adds source-control discipline as an operating layer, not as an automation layer.

```
state -> graph -> packet -> checkpoint -> review/export -> guided workflow -> agent driver -> management/recovery -> design guidance -> operating discipline -> source-control discipline
```

The source of truth remains .aiqt/project.json, .aiqt/state.json, and .aiqt/runlog.jsonl. Generated exports remain non-canonical. M15 must not turn AIQT into a Git automation tool or GitHub client.

2. Milestone Summary

M15 hardens AIQT so coding agents treat the implementation repository as a controlled software project. It adds guidance for Git initialization, main-branch baseline, per-work-unit commits, per-work-unit tags, GitHub flow, and self-attested risk reporting.

3. Findings Addressed

Finding	Problem	M15 response
F058	Dogfood implementation repositories were not consistently source controlled.	Add repository/source-control discipline into driver, plan, packet, checkpoint, and audit metadata.
F059	No default-branch contract.	Require main as the implementation repository default branch unless explicitly overridden.
F060	No per-work-unit commit/tag boundary.	Require guidance for one detailed commit and one tag after every completed Work Unit.
F061	No Work Unit risk score in commits/checkpoints.	Require 0-100 risk score mapped to canonical AIQT severity.
F062	GitHub flow not exercised before packaging.	Add branch/remote/PR/push/tag guidance without GitHub automation.

4. Scope

4.1 In scope

- Driver prompt source-control discipline.
- Planning prompt requirement to include repository initialization/baseline work in new implementation plans.
- Agent packet Source Control Expectations section.
- Checkpoint prompt source-control/risk reporting guidance using existing checkpoint fields.
- GitHub flow guidance for human/agent-assisted push/PR/review flow.
- Packet audit metadata flag includesSourceControlGuidance, stored in agent_packet.created runlog event data and mirrored in state.lastAgentPacket when applicable.
- Risk score to canonical severity mapping.

- Tests proving no new commands and no exit-code drift.

4.2 Out of scope

- AIQT directly executing git commands.
- Automatic commits or automatic tags.
- GitHub API, gh CLI, remote repository creation, or branch protection automation.
- Credential handling for GitHub.
- Parsing, verifying, or enforcing commit hashes/tags/branch state.
- Checkpoint schema changes.
- AIQT_SCHEMA_VERSION bump.

5. Behavioral Position

M15 is a guidance and audit-signal milestone. AIQT instructs the coding agent how source control should be handled, but the agent or user executes Git/GitHub actions. AIQT must not claim verification it did not perform.

All Git/GitHub status reported through checkpoints is self-attested by the agent as formatted text inside existing checkpoint summary and/or notes fields. AIQT does not validate the truth of that report.

6. Repository Initialization Contract

6.1 Root model

- AIQT control root: directory containing .aiqt/.
- Implementation root: application source repository. This should be a Git repository.
- Source-control discipline applies to the implementation root, not necessarily the AIQT control root.
- If the roots differ, M14 working-directory discipline still applies.

6.2 Default branch

The implementation repository default branch should be main unless the user explicitly specifies another branch policy.

```
git init -b main
# fallback if needed:
git init
git branch -M main
```

6.3 Baseline commit

If the implementation root is not a Git repository, the driver/plan guidance should require repository initialization before feature work. The baseline should include .gitignore and the scaffold state before Work Unit implementation starts.

```
chore(repo): initialize implementation repository baseline
```

```
AIQT Repository Baseline
Implementation root: <path>
Default branch: main
Baseline contents:
- project scaffold or empty implementation root
- .gitignore configured
- source-control discipline established
```

```
Risk score: 0/100
AIQT severity: low
Risk rationale: Repository initialization only. No application behavior implemented yet.
```

Suggested baseline tag: repo-baseline.

7. Per-Work Unit Commit and Tag Contract

After every Work Unit successfully reaches a checkpoint, the implementation repository should receive exactly one detailed Work Unit commit and one Work Unit tag, unless the user explicitly instructs otherwise or the repository is intentionally not source controlled.

7.1 Commit message template

```
feat(WU003): implement product profile model and form

AIQT Work Unit:
WU003 - Product profile model and form

Milestone:
M002 - Product Profile and Local State

Summary:
- Added product profile domain model.
- Implemented editable product profile form.
- Added validation for required fields.
- Connected form state to persistence.

Files changed:
- src/domain/product-profile.ts
- src/components/product/ProductProfileForm.tsx
- tests/product-profile.test.ts

Validation:
- npm run lint: passed
- npm run build: passed
- npm run test: passed
- Playwright browser check: passed

Checkpoint result:
- Acceptance criteria: passed
- Validation: passed
- Status: done

Risk score: 12/100
AIQT severity: low
Risk rationale:
Low risk. Changes are isolated to product profile state and form UI.

Known limitations:
- No backend persistence by design.
```

7.2 Tag format

Use a deterministic tag after each Work Unit commit. Recommended format:

```
m###-wu###-done
```

Examples:

```
m001-wu001-done
```

```
m002-wu003-done
```

For dogfood applications, lightweight tags are acceptable. For AIQT's own repository, annotated tags remain preferred when release semantics matter.

8. Risk Score Contract and Severity Mapping

The 0-100 Work Unit risk score is retained because the user explicitly wants a precise risk number in each Work Unit commit/checkpoint. However, AIQT already has a canonical severity taxonomy: low, medium, high, critical. M15 must not create a disconnected parallel severity model.

The numeric score is an agent-facing precision layer. The categorical severity remains AIQT's canonical severity taxonomy. When both are present, the score must map deterministically to the canonical severity.

Risk score	Canonical AIQT severity	Meaning
0-20	low	Low implementation or release risk.
21-50	medium	Meaningful but manageable risk.

51-80	high	Serious unresolved risk, incomplete verification, or significant uncertainty.
81-100	critical	Blocked, unsafe to treat as complete, or critical release risk.

If a Work Unit risk score maps to high or critical severity, the agent should normally checkpoint as `needs_review` unless the risk is explicitly external, user-action-required, or accepted by the user. AIQT does not automatically enforce this rule in M15.

9. Self-Attested Reporting Rule

M15 does not add a checkpoint schema change. Therefore, source-control and risk reporting must be self-attested by the agent in existing checkpoint summary and/or notes fields.

- AIQT does not execute git commands.
- AIQT does not verify commit hashes.
- AIQT does not verify tag existence.
- AIQT does not verify GitHub remote/PR state.
- AIQT does not parse or enforce the risk score.
- AIQT does not automatically change checkpoint status based on risk score.
- The guidance is still required in prompts/packets so agents report it consistently.

10. GitHub Flow Guidance

M15 adds guidance for GitHub flow without integration automation. If a GitHub remote exists or the user asks for GitHub flow, the agent should follow a controlled branch/commit/push/PR discipline.

- Default branch remains main.
- Feature work may use work-unit branches when appropriate, but dogfood may also commit directly to main if explicitly chosen for local speed.
- Remote push and PR creation are user/agent actions, not AIQT actions.
- Do not claim PR status, checks, merge state, or branch protection unless actually verified by the agent/user.
- Tags should be pushed only when the user or dogfood protocol requires it.

11. Prompt and Packet Surfaces

11.1 Driver prompt

aiqt prompt driver should include Source Control Discipline. It should instruct the agent to confirm the implementation root, initialize Git if missing, ensure main as default branch, create/update .gitignore, create a baseline commit, run git status before each Work Unit, and commit/tag after each completed Work Unit.

11.2 Planning prompt

aiqt prompt plan should require a repository initialization/baseline Work Unit for new implementation projects unless source control already exists or the user explicitly disables it. This work should appear before feature implementation.

11.3 Next packet

aiqt next should include a Source Control Expectations section for implementation Work Units.

11.4 Checkpoint prompt

aiqt prompt checkpoint should ask the agent to report source-control and risk details using existing checkpoint fields.

```
Source control report:
- Implementation root:
- Current branch:
- Git status before checkpoint:
```

```

- Commit created: yes/no
- Commit hash:
- Tag created: yes/no
- Tag name:
- Uncommitted files remaining:
- Ignored/generated artifacts:

Risk report:
- Risk score: <0-100>/100
- AIQT severity: low|medium|high|critical
- Risk rationale:
- Release/user-action/external verification gaps:

```

12. Deterministic Packet Section Ordering

Packet section order must remain deterministic and testable. M15 adds Source Control Expectations without leaving render order to implementation choice.

Current packet order should be:

1. Scope
2. Out of Scope
3. Context
4. Constraints
5. Dependencies
6. Acceptance Criteria
7. Validation
8. Working Directory Discipline
9. Component-System Guidance
10. Design Guidance
11. Source Control Expectations
12. Relevant Skills
13. Required Output

If the current repository template has a different established canonical order, the implementation may preserve that order, but Source Control Expectations must render in a fixed location and must be covered by packet audit metadata.

13. Packet Audit Metadata

M15 extends the M14 packet audit model additively. The authoritative historical audit record is the `agent_packet.created` runlog event data. `state.lastAgentPacket` may mirror the latest values only.

```

{
  "eventType": "agent_packet.created",
  "data": {
    "packetId": "PKT-015",
    "workUnitId": "WU015",
    "renderedSections": [
      "scope",
      "context",
      "validation",
      "workingDirectoryDiscipline",
      "componentSystemGuidance",
      "designGuidance",
      "sourceControlExpectations"
    ],
    "guidanceFlags": {
      "includesWorkingDirectoryDiscipline": true,
      "includesComponentSystemGuidance": true,
      "includesDesignGuidance": true,
      "includesSourceControlGuidance": true
    }
  }
}

```

No new runlog event type is required. The agent-packet export should show whether source-control guidance was present when historical metadata is available.

14. Repository Implementation Scope

Implementation should modify the existing prompt templates, packet context/template, packet audit metadata computation, runlog event data type, export rendering, and tests. The file map below is descriptive, not mandatory.

- Modify the existing AgentPacketMetadataSchema location, wherever it currently lives. Do not create a new schema file solely to satisfy this specification.
- Modify the existing runlog event data typing for agent_packet.created, wherever it currently lives.
- Modify existing driver/plan/checkpoint prompt templates.
- Modify existing agent packet rendering and audit metadata code.
- Modify existing agent-packet export rendering.
- Add focused unit/integration tests.

15. Exit Codes and Commands

M15 must not add new commands or alter exit-code behavior. Existing command success/failure behavior should remain unchanged. Tests should prove top-level command registration is unchanged and representative pre-M15 exit-code contracts still hold.

16. Testing Scope

16.1 Unit tests

- Risk score to severity mapping helper, if implemented.
- Source-control guidance rendering text.
- Tag/commit template rendering text.
- Packet audit metadata includes includesSourceControlGuidance when Source Control Expectations renders.

16.2 Integration tests

- aiqt prompt driver includes source-control discipline.
- aiqt prompt plan requires repository initialization/baseline for implementation projects.
- aiqt prompt checkpoint asks for source-control/risk report.
- aiqt next packet renders Source Control Expectations in deterministic order.
- agent_packet.created runlog event data includes source-control audit flag when section renders.
- aiqt export agent-packet displays source-control guidance audit metadata.
- No new commands are registered.
- No exit-code drift for representative existing commands.

17. Validation Commands

Run the normal validation suite. If pnpm is blocked by the known ignored-builds/esbuild gate, use the direct fallback and report the pnpm limitation explicitly.

```
pnpm validate

# fallback if pnpm validate is blocked:
node_modules/.bin/tsc --noEmit
node_modules/.bin/eslint .
node_modules/.bin/vitest run
node_modules/.bin/tsc
```

18. Definition of Done

- M15 guidance appears in driver, plan, packet, and checkpoint prompt surfaces.
- Source Control Expectations renders deterministically in packets.

- Packet audit metadata includes includesSourceControlGuidance in agent_packet.created runlog data when applicable.
- state.lastAgentPacket mirrors latest audit metadata only; runlog remains authoritative history.
- Risk score maps to canonical low/medium/high/critical severity.
- The spec does not introduce direct Git/GitHub automation.
- No new commands, no schema version bump, no new runlog event types.
- Tests pass through direct validation fallback if pnpm validate is blocked.

19. Required Implementation Report

- Branch name and commit hash.
- Files changed summary.
- Source files created/modified.
- Tests created/modified and final test count.
- Whether schema changed and whether AIQT_SCHEMA_VERSION changed.
- Whether any command was added or exit-code behavior changed.
- How source-control guidance is rendered in driver/plan/packet/checkpoint surfaces.
- How packet audit metadata is stored in runlog and mirrored in state.
- How risk score maps to canonical AIQT severity.
- Validation results including pnpm validate or direct fallback.
- Deviations from spec.
- Working tree status after commit.

20. Dogfood v4 Acceptance Target

After M15, dogfood v4 should test the entire AIQT flow with an intermediate/full-stack project and GitHub flow. The target app should be more realistic than prior local-only dogfoods.

- Next.js App Router, TypeScript, Tailwind, shadcn/ui.
- Clerk authentication.
- Supabase database and RLS assumptions.
- Supabase Storage or safe placeholder if credentials are not available.
- Explicit AIQT control root and implementation root.
- Implementation repository initialized on main.
- Baseline commit before feature work.
- Commit and tag after every completed Work Unit.
- Work Unit commit/checkpoint risk score and canonical severity.
- GitHub remote/branch/PR flow if available; otherwise explicit user-action-required gap.
- Use AIQT issue lifecycle, graph validation/repair, checkpoint amendment if needed, final review, manage, and export all.

21. Claude Review v0.1 Corrections Applied

- Risk score now maps deterministically to canonical AIQT severity.
- Self-attested source-control/risk reporting is explicitly stated and not presented as AIQT-verified.
- Packet section ordering is explicit.
- Source Alignment section added.

- Schema file paths are descriptive, not mandatory.
- No new command/no automation scope remains intact.